41039 Programming 1

Revision of Week 11: Designing object-oriented programs



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

How the revision session works

A recap before the next lecture

Write an answer before the reveal. Explain the state change, rule or contract that makes it follow. These are practice questions, with no new assessment requirements.

Part 1: on your own

- 6 short questions
- About 90 seconds per question
- Java and Python; name the language first

Part 2: teams of 3 or 4

- Draw the model
- Spot the errors
- Predict the output
- Write and test the code

A result needs a reason. A repair needs a counterexample.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

1. Keep the calculation in the expert

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Order order=new Order();
order.add(new Line(300,2));
order.add(new Line(150,1));
System.out.println(order.total());
class Line {
    private final int cents,quantity;
    Line(int cents,int quantity) { this.cents=cents;
     this.quantity=quantity; }
    int subtotal() { return cents*quantity; }
}
class Order {
    private final java.util.List<Line> lines=new java.
     util.ArrayList<Line>();
    void add(Line line) { lines.add(line); }
    int total() { int sum=0; for(Line line:lines) sum+=
     line.subtotal(); return sum; }
}
```

Your answer

Predict the total. Which object should compute one line subtotal, and which object should aggregate the order?

1. Keep the calculation in the expert

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Order order=new Order();
order.add(new Line(300,2));
order.add(new Line(150,1));
System.out.println(order.total());
class Line {
    private final int cents,quantity;
    Line(int cents,int quantity) { this.cents=cents;
     this.quantity=quantity; }
    int subtotal() { return cents*quantity; }
}
class Order {
    private final java.util.List<Line> lines=new java.
     util.ArrayList<Line>();
    void add(Line line) { lines.add(line); }
    int total() { int sum=0; for(Line line:lines) sum+=
     line.subtotal(); return sum; }
}
```

Answer

750

Each line computes its own subtotal; Order sums 600 and 150.

2. Separate calculation from presentation

Python 3; commit to an outcome before the reveal.

```
class Order:
    def __init__(self, amounts): self.amounts=list(
     amounts)
    def total(self): return sum(self.amounts)
def format_receipt(order):
    return f"Total: {order.total()} cents"
order=Order([600,150])
print(order.total())
print(format_receipt(order))
```

Your answer

What is the complete output? Which line or rule decides it?

2. Separate calculation from presentation

Python 3; commit to an outcome before the reveal.

```
class Order:
    def __init__(self, amounts): self.amounts=list(
     amounts)
    def total(self): return sum(self.amounts)
def format_receipt(order):
    return f"Total: {order.total()} cents"
order=Order([600,150])
print(order.total())
print(format_receipt(order))
```

Answer

```
750
Total: 750 cents
```

The calculation returns a number; the formatter builds a human-readable receipt from that result.

3. An extensible pricing rule

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Basket basket=new Basket(new StudentPrice());
System.out.println(basket.price(1000));
interface PriceRule { int apply(int cents); }
class StudentPrice implements PriceRule {
    public int apply(int cents) { return cents*80/100; }
}
class Basket {
    private final PriceRule rule;
    Basket(PriceRule rule) { this.rule=rule; }
    int price(int cents) { return rule.apply(cents); }
}
```

Your answer

What prints? Which parts depend on the pricing interface, and how does that help when adding a new policy?

3. An extensible pricing rule

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Basket basket=new Basket(new StudentPrice());
System.out.println(basket.price(1000));
interface PriceRule { int apply(int cents); }
class StudentPrice implements PriceRule {
    public int apply(int cents) { return cents*80/100; }
}
class Basket {
    private final PriceRule rule;
    Basket(PriceRule rule) { this.rule=rule; }
    int price(int cents) { return rule.apply(cents); }
}
```

Answer

800

StudentPrice applies the 80% policy. Basket depends on the PriceRule abstraction and receives its collaborator through the constructor. This supports dependency inversion and lets a new policy extend the behaviour without changing Basket.price.

4. Inject a function in Python

Python 3; commit to an outcome before the reveal.

```
def student_price(cents): return cents*80//100
def regular_price(cents): return cents
class Basket:
    def __init__(self, rule): self.rule=rule
    def price(self, cents): return self.rule(cents)
print(Basket(student_price).price(1000))
print(Basket(regular_price).price(1000))
```

Your answer

What is the complete output? Which line or rule decides it?

4. Inject a function in Python

Python 3; commit to an outcome before the reveal.

```
def student_price(cents): return cents*80//100
def regular_price(cents): return cents
class Basket:
    def __init__(self, rule): self.rule=rule
    def price(self, cents): return self.rule(cents)
print(Basket(student_price).price(1000))
print(Basket(regular_price).price(1000))
```

Answer

```
800
1000
```

The caller supplies different policies to the same Basket implementation.

5. Substitutability includes behaviour

Java 8; commit to an outcome before the reveal.

```
public class Demo {
    static void fill(java.util.List<Integer> xs) { xs.
     add(2); xs.add(3); }
    public static void main(String[] args) {
        java.util.List<Integer> a=new java.util.
     ArrayList<Integer>();
        java.util.List<Integer> b=new java.util.
     LinkedList<Integer>();
        fill(a); fill(b);
        System.out.println(a.equals(b));
        System.out.println(a.get(0));
    }
}
```

Your answer

Predict the output. Would a replacement List remain substitutable if its add method inserted at the front?

5. Substitutability includes behaviour

Java 8; commit to an outcome before the reveal.

```
public class Demo {
    static void fill(java.util.List<Integer> xs) { xs.
     add(2); xs.add(3); }
    public static void main(String[] args) {
        java.util.List<Integer> a=new java.util.
     ArrayList<Integer>();
        java.util.List<Integer> b=new java.util.
     LinkedList<Integer>();
        fill(a); fill(b);
        System.out.println(a.equals(b));
        System.out.println(a.get(0));
    }
}
```

Answer

```
true
2
```

Both implementations preserve append order, so their contents are equal and begin with 2. A replacement that prepended instead would violate the List contract. Matching a method signature does not prove behavioural substitutability.

6. A narrow capability

Python 3; commit to an outcome before the reveal.

```
class Screen:
    def send(self, message): return "screen: "+message
class Recorder:
    def __init__(self): self.messages=[]
    def send(self, message):
        self.messages.append(message)
        return "recorded"
def notify(channel, message): return channel.send(
     message)
recorder=Recorder()
print(notify(recorder,"Ready"))
print(recorder.messages)
```

Your answer

What is the complete output? Which line or rule decides it?

6. A narrow capability

Python 3; commit to an outcome before the reveal.

```
class Screen:
    def send(self, message): return "screen: "+message
class Recorder:
    def __init__(self): self.messages=[]
    def send(self, message):
        self.messages.append(message)
        return "recorded"
def notify(channel, message): return channel.send(
    message)
recorder=Recorder()
print(notify(recorder,"Ready"))
print(recorder.messages)
```

Answer

```
recorded
['Ready']
```

The recorder fulfils the same send capability and stores the message for inspection.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Form teams

- Teams of 3 or 4. One shared sheet, one spokesperson.
- For each challenge, record the result and the reason.
- Rotate the spokesperson. Other teams can challenge an explanation.
- The final task has starter files and a Java/Python solution pair.

Discuss first. Commit to an answer. Then compare with the reveal.

Draw it: A responsibility map for an order

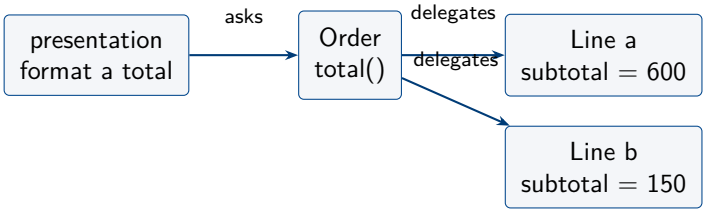
```
Line a=new Line(300,2);
Line b=new Line(150,1);
Order order=new Order(a,b);
System.out.println(order.total());
class Line {
    private final int cents,quantity;
    Line(int cents,int quantity) { this.cents=cents;
     this.quantity=quantity; }
    int subtotal() { return cents*quantity; }
}
class Order {
    private final Line a,b;
    Order(Line a,Line b) { this.a=a; this.b=b; }
    int total() { return a.subtotal()+b.subtotal(); }
}
```

Course website: Week 11. Complete programs accompany this deck.

Four minutes

Draw the flow and responsibilities after the shown operations. Label the important state. Mark what is shared, what changes, or which path handles failure. Predict every output line.

Draw it: one answer



Output

```
750
```

Line is the information expert for its own subtotal. Order aggregates its lines. Presentation can change its wording without changing the arithmetic.

Course website: Week 11. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Spot the errors 1: Java (5 min)

Contract

With HalfPrice, total(1000) returns 500 without printing. Main prints one line.

```java
public class Broken {
    public static void main(String[] args) {
        Checkout c=new Checkout(new HalfPrice());
        System.out.println(c.total(1000));
    }
}
interface Pricing { int apply(int cents); }
class Regular implements Pricing {
    public int apply(int cents) { return cents; }
}
class HalfPrice implements Pricing {
    public int apply(int cents) { return cents/2; }
}
class Checkout {
    private final Pricing rule;
    Checkout(Pricing rule) { this.rule=new Regular(); }
    int total(int cents) {
        System.out.println("charging");
        return rule.apply(cents);
    }
}
```

Spot the errors 1: answers

The repairs

- **L16:** Keep the supplied Pricing collaborator instead of constructing an unrelated fixed policy.
- **L18:** A calculation-only total method should return the charge without choosing a presentation side effect.

Course website: Week 11. Complete programs accompany this deck.

✗ Original program

```
charging
1000
```

✓ After the repairs

```
500
```

Spot the errors 2: Python (5 min)

Contract

Count and labels must agree at the boundary, and one function must own the pass rule.

```python
def passes(mark): return mark>50

def count_passes(marks):
    return sum(passes(mark) for mark in marks)
def labels(marks):
    return ["P" if mark>=50 else "F" for mark in marks]

marks=[49,50,85]
print(count_passes(marks))
print(labels(marks))
```

Spot the errors 2: answers

The repairs

- **L1:** Include the exact pass boundary: 50 passes.
- **L6:** Use passes(mark) here too, so counting and labels share one source of the pass rule.

✗ Original program

```
1
['F', 'P', 'P']
```

✓ After the repairs

```
2
['F', 'P', 'P']
```

Course website: Week 11. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Predict 1: A policy and a common caller

```
public class Demo {
    static int charge(Price rule,int cents) { return
     rule.apply(cents); }
    public static void main(String[] args) {
        Price[] rules={new Regular(),new Half()};
        for(Price rule:rules) System.out.println(charge(
     rule,101));
    }
}
interface Price { int apply(int cents); }
class Regular implements Price {
    public int apply(int cents) { return cents; }
}
class Half implements Price {
    public int apply(int cents) { return cents/2; }
}
```

Course website: Week 11. Complete programs accompany this deck.

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Predict 1: answer

```
101
50
```

Why

charge depends only on Price. The two objects select different policies; integer division truncates the fractional cent for Half.

Course website: Week 11. Complete programs accompany this deck.

Predict 2: One capability, observable interactions

```
class Recorder:
    def __init__(self): self.messages=[]
    def send(self,text):
        self.messages.append(text)
        return len(self.messages)
def notify(channel,text): return channel.send(text)
a=Recorder()
b=a
print(notify(b,"Ready"))
print(notify(a,"Go"))
print(a.messages,b.messages is a.messages)
```

Course website: Week 11. Complete programs accompany this deck.

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Predict 2: answer

```
1
2
['Ready', 'Go'] True
```

Why

The recorder is a test collaborator. Both variables identify one recorder, so both requests add to the same history.

Course website: Week 11. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Code it: specification

Information experts in an order

Read n, then n pairs: non-negative unit price in cents and quantity. Line.subtotal computes the product. Order.total sums the line subtotals. Print the total. Explain which responsibility belongs to each class.

Example input

```
2
300 2
150 1
```

Expected output

```
750
```

Eight minutes. One Java team and one Python team at the keyboard. Other teams produce a boundary test and explain which wrong implementation it would expose.

Code it: Java solution

```java
import java.util.*;
Scanner sc=new Scanner(System.in);
int n=sc.nextInt();
Order order=new Order();
for(int i=0;i<n;i++) order.add(new Line(sc.nextInt(),sc.nextInt()));
System.out.println(order.total());
class Line {
    private final int price,quantity;
    Line(int price,int quantity) { this.price=price; this.quantity=quantity; }
    int subtotal() { return price*quantity; }
}
class Order {
    private final List<Line> lines=new ArrayList<Line>();
    void add(Line line) { lines.add(line); }
    int total() { int total=0; for(Line line:lines) total+=line.subtotal(); return total; }
}
```

Course website: Week 11. Complete programs accompany this deck.

Code it: Python solution

```python
class Line:
    def __init__(self,price,quantity): self.price,self.quantity=price,quantity
    def subtotal(self): return self.price*self.quantity
class Order:
    def __init__(self): self.lines=[]
    def add(self,line): self.lines.append(line)
    def total(self): return sum(line.subtotal() for line in self.lines)
order=Order()
n=int(input())
for _ in range(n):
    price,quantity=map(int,input().split())
    order.add(Line(price,quantity))
print(order.total())
```

Course website: Week 11. Complete programs accompany this deck.

Code it: the tests

A slash separates input lines; (blank) denotes an empty line.

Test 1
`2 / 300 2 / 150 1`
Output: 750

Test 2
0
Output: 0

Test 3
`3 / 0 10 / 100 0 / 1 1`
Output: 1

Explain why each test is present, not only whether it passes.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

What you should now be able to explain

- Assign behaviour to the object with the needed information.
- Keep computation independent of display and file choices.
- Supply collaborators through a useful abstraction.
- Use SOLID, GRASP, DRY and YAGNI to justify concrete decisions.

Return to the matching week page for runnable examples and further practice. Bring one unresolved prediction or counterexample to the next class.
Course website: Week 11. Complete programs accompany this deck.